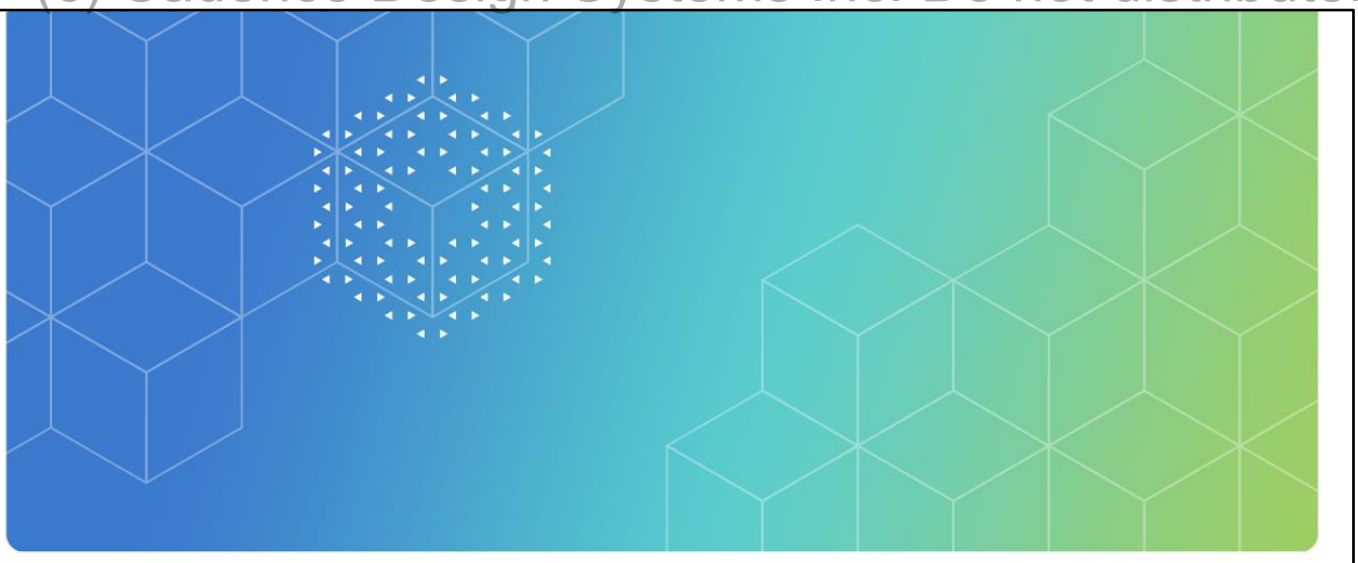


Module 18

Simulation Automation and Scripting

cadence

This page does not contain notes.

Submodule Objectives

In this submodule, you will:

- Identify the need for and benefits of automation through scripting.
- Automate compilation through run files.
- Automate simulation through Tcl commands.
- Create your own Tcl scripts.
- Identify the role of simulation management tools.

This page does not contain notes.

Automation and Scripting

Verification requires a huge number of simulation runs

There are a large number of variables and parameters to control

- For simulation, e.g. randomization seeds, compile and elaboration options, design parameters
- For debug, e.g. breakpoints, selected signals for waveforms, and watchlists

We use command scripts to control simulation, allowing

- Repeated simulation runs with consistent parameters/options

We use automation to schedule and execute multiple simulation runs, allowing

- Automatic test selection
- Collation and analysis of results, coverage, and debug data
- Tracking of data over time

cadence

This page does not contain notes.

Example Uses for Automation in Verification

Layer	Purposes	Implementation
Compilation	- Correct compilation - File dependency management - Consistent use of tool options	- run files - makefiles - shell scripts
Simulation	- Reproducible simulation runs - Consistent simulation environment	- Tcl/Python - shell scripts
Debug	- Waveform and transaction tracing - Breakpoint setup - Log files - Custom debugging features	- Tcl/Python
Verification Management	- Verification planning - Job scheduling & execution - Test management - Coverage collation	- GUI on database languages - Python APIs

cadence

This page does not contain notes.

Compilation Run Files

dut1_run.f

```
/*------------------------------
Description   : UVCVIF simulation run file
Created       : 01/04/20
-----------------------------------------------*/
-uvmhome $UVMHOME

//Simulator options
+SVSEED=random
-timescale 1ns/1ns
+access+rwc

// Add directories to reference search path
-incdir ../sv

// TB files
../sv/yapp_pkg.sv
../sv/yapp_if.sv
tb_top.sv

// TB options
+UVM_VERBOSITY=UVM_HIGH
+UVM_TESTNAME=exhaustive_seq_test

// DUT files
$ROUTERDUT/router_rtl/yapp_router.sv
```

A text file, typically including:

- Compiler and simulator options
- Testbench and DUT files in correct order
- Testbench options
- Comments

Filename is referenced on simulator invocation

- Alongside additional options
- For example:
  - `xrun **-f dut1_run.f** -gui`

Portability guidelines should be followed, e.g.

- Use relative, not absolute, pathnames
- `incdir` compiler options help resolve references to files in other or remote directories

cadence

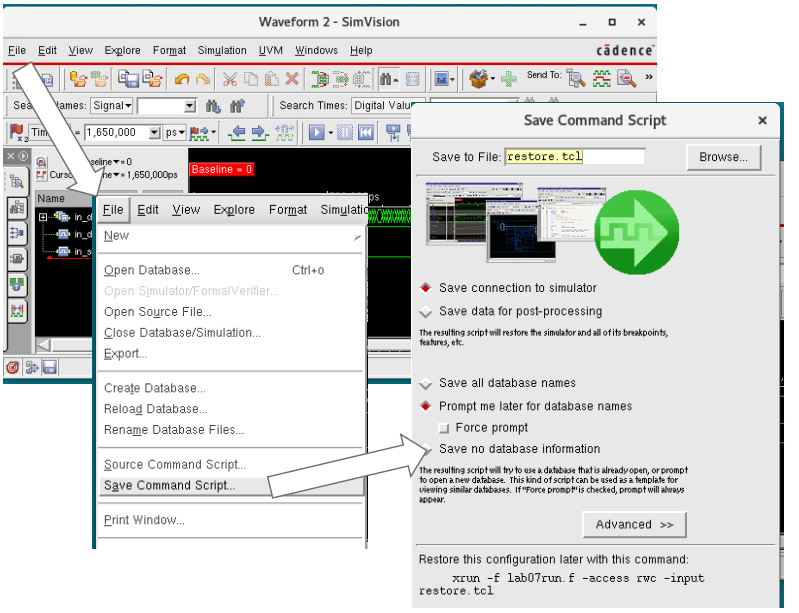
A simple example of automation scripting is a simulator run file. This is a simple text file that lists the testbench and DUT files for simulation in the correct compilation order. The file also typically contains compiler and simulator tool command options, testbench options, such as the selection of a specific test, and embedded comments. The run file is referenced on the simulator command line, alongside other command options. Effectively the reference is replaced by a copy-paste of the run file contents.

A run file saves us from having to type out all the filenames and options in full for every simulator run.

Run files should be written for portability, so if the simulation files are moved from one location to another, the run file is still valid. For example, you should avoid absolute hierarchical pathnames in the file. Relative pathnames, or pathnames containing environmental variables, are more robust if the files are moved. Specific compilation options, e.g. incdir for the Xcelium simulator, give the compiler a list of directories to search for the resolution of file references within the testbench or DUT code.

More complex compilation scripts may use software-like makefiles or shell scripts. These are useful if you need to execute shell or operating system commands as part of running a simulation. For example, to set environmental variables, create directories, or check for the existence of additional executables.

Simulation Setup



Setup can be saved and sourced

Typically, divided into two:

- Simulator environment
  - Creates the results database
  - Includes probes, breakpoints etc.
- Debugging environment
  - Views the results database
  - Includes waveform signals, markers, groups, mnemonics etc.

Allows consistent debugging over repeated and multiple simulations

6

cadence

Using the full power of a debugging environment usually means a complex setup of specific signals in specific groups and in a specific order in the waveform viewer, supplemented by breakpoints, markers, and mnemonics to understand the traced data.

Typically, you want to reuse this setup for further simulations using the same or different test stimulus, without having to build the environment from scratch each time.

A simulator will provide a means to save the existing environment so you can easily restore the setup in subsequent simulations. For example, Xcelium SimVision has save and source command script options.

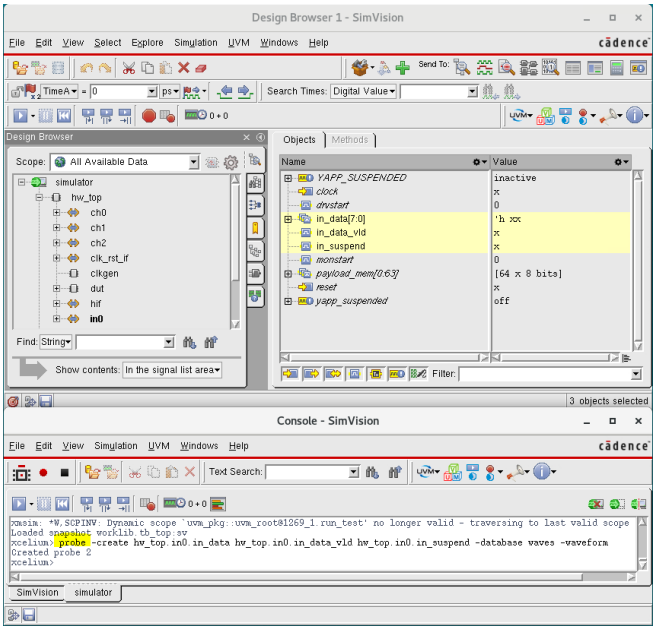
Typically, there are two command scripts:

Simulator environment – these options create the results database and include traced signals (probes), breakpoints, forces and deposits etc.

Debugging environment – these options control how the results are displayed and include the order of signals in the waveform viewer, groups of signals, timeline markers, and mnemonics to help understand data. These options also control the layout and preferences for debugging windows.

These options allow an environment to be reused over multiple simulations, even with different stimulus data, allowing a consistent debugging environment. They also allow personalization of the simulator environment, as well as design, system and project specific setups.

Simulator Run Scripts



- Simulator GUI operations are reflected as commands in the console window
- Commands can be copied/saved from console
  - Use directly or edit as simulation script
- Scripts can be invoked from the console
  - `source myscript.tcl`
- Or added to command line to run automatically
  - `xrun ... -input myscript.tcl`
- Commands use Tool Command Language (Tcl) syntax
  - Although some newer tools support Python

cadence

This page does not contain notes.

Tcl (Tool Command Language)

High-level, general-purpose, compact programming language

- Used by most EDA tools for scripting

Characteristics:

- Interpreted
  - Code is directly executed without the need for compilation
- Case-sensitive
- Everything is a command
  - Variables are not declared but simply assigned
- Every command is terminated by a semi-colon (`;`) or new-line
- All variables are strings...
  - ... but strings can represent integer, real, Boolean values...
  - ... and also data structures such as lists and arrays
- Comments begin with `#` and end with a new-line

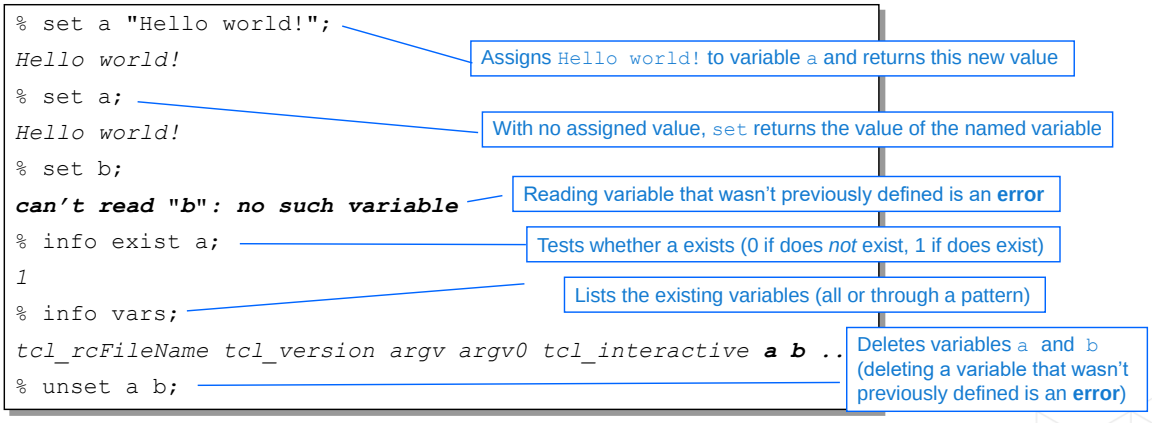
cadence

This page does not contain notes.

Tcl Variables

There is no need to declare variables.

- Use **`set`** to create a variable and assign/retrieve its value.
- Use **`unset`** to delete one or more variables.
- Tcl is typeless – variables store STRING values.



cadence

Syntax

```
set varName ?value?
unset varName ?varName ...?
```

Examples

```
% unset doesnoexist
can't unset "doesnoexist": no such variable
% info exist b
0
% info vars ?
a
```

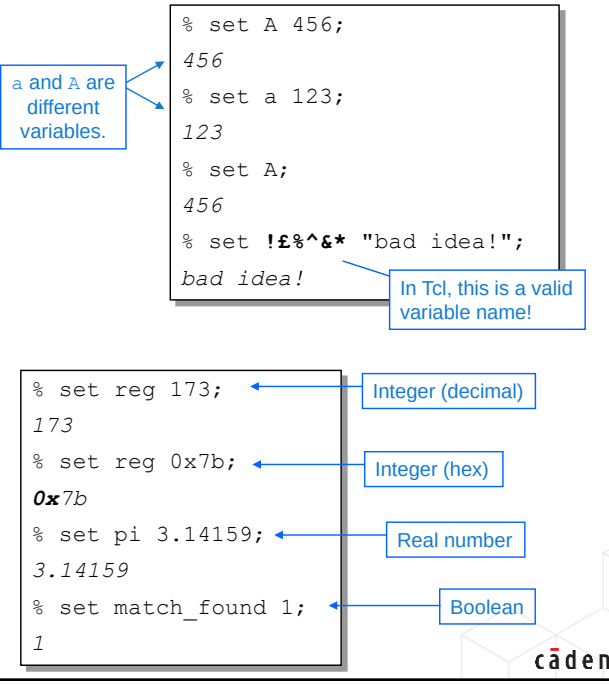
Variable Names and Values

Variable names:

- Are case-sensitive
- May be composed of any characters
  - Use only letters, digits, and underscores in variable names to maintain readability!

Strings can also represent numeric values

- **Integers:** `123` (decimal), `0x7b` or `0x7B` (hexadecimal), `0173` (octal)
- **Real numbers:** `3.14159`, `1.23e+2` or `1.23E2`, `123.0`
- **Boolean:** `0` (false), `1` (true)



cadence

This page does not contain notes.

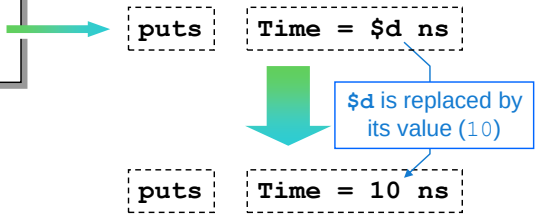
Variable Substitution ($)

Each occurrence of `$varName` is replaced with the corresponding variable value:

- Except where `$` is escaped (`\$`).

`puts` writes to the shell

```
% set d 10;
10
% puts "Time = d ns"
Time = d ns
% puts "Time = $d ns"
Time = 10 ns
```



11

cadence

This page does not contain notes.

Backslash Substitution

Backslash substitution can be used to insert:

- Special terminal characters:
  - **\n** (newline), **\t** (tab), **\b** (backspace), **\a** (audible alert), ...
- Any character using its binary value:
  - **\x**4f or **\x**4F (hexadecimal values), \117 (octal value)
- Escape special characters or just replicate other characters:
  - **\$** (dollar), **** (backslash), **\"** (quotes), **\}** (brace), ****□ (space)

```
% set Flag rst; puts "Flag\t\"\$Flag\" on";
Flag    "$Flag" on
% puts "Synthesis done. \a\a\a";
Synthesis done.
% puts "ABC = \x41\102\x43";
ABC = ABC
% puts "Some specials: \$\"\\";
Some specials: $"\
```

Beeps 3 times

Inserting binary values into a Tcl string

Escaping special characters $, " and \

cadence

This page does not contain notes.

Command Substitution ([])

[<command>] is replaced with the result of executing <command>

- Or the last command if [] contains multiple commands
- Unless [] are escaped (\[and \])

Contents of [] must be a command

- Otherwise [and] are treated as part of the string

```
% set d 10;
10
% puts "Time = [set d]ns";
Time = 10ns
% puts "Time = $d ns";
Time = 10 ns
% set sum [1+2+3];
[1+2+3]
```

Note extra space to delimit variable

This is a string, not a command! (See next slide)

cadence

When the Tcl interpreter encounters a closing square bracket “]”, it evaluates whatever is within the matching two square brackets “[“ ”]” and evaluates it as a command. Then, it replaces the full content of the square brackets with the result of the command (or the result of the last command if there are several commands separated with a semi-column “;”) and proceeds further.

Examples

```
% set d 10
% set e 12
% puts “Time = [set d ; set e]ns”
Time = 12ns
```

Expressions

Use the `expr` command to evaluate mathematical expressions.

- Treats text string as an expression

Enclose expression in braces `{}` if it contains variables

- More efficient
  - Variable substitution is done once by `expr` rather than first by the interpreter and then by `expr`

Braces control evaluation of expression, e.g.

- `[expr ($a == 1) || ($a == 2)];`

Use real number strings to force real number operators

```
% set sum 1+2+3+4;
1+2+3+4
% set sum [1+2+3+4];
[1+2+3+4]
set sum [expr 1+2+3+4];
10
% set a 10; set b 20;
20
% set sub [expr $a - $b];
-10
% set sub [expr {$a - $b}];
-10
% set div [expr 9/2];
4
% set div [expr 9/2.0];
4.5
```

Expression treated as a string

`expr` evaluates the expression

Enclose the expression in braces for efficiency

Integer arithmetic

Real number arithmetic

`||` is the logical OR operator. == is the equality operator. Note the symbol = has no special meaning in Tcl.

Substitution Example

```
% set a 3
% set b {$a + 2}
% expr $b*4
11
```

Why does the final expression return 11? Well, the expression is substituted twice: once by the Tcl parser and once by the `expr` command, so the result of the expression is `$a+2*4`

The improved speed of calculation when the expression is enclosed in braces can be observed even in the simple example above.

`expr {$a -$b}` executes **7 times faster** than `expr $a - $b`

Conditional Execution: `if-elseif-else`

Execute commands IF condition is true:

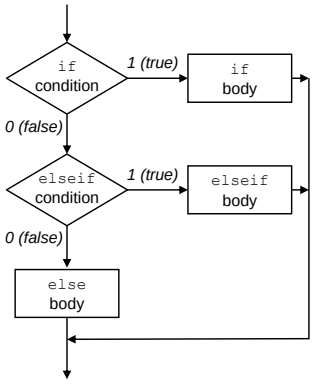
- Condition is evaluated in the same way as `expr` expression
- Enclose conditions and `if`/`elseif`/`else` command bodies in `{}`

Optionally test for more than one condition with `elseif`

- Any number of `elseif`s can be used

Optional unconditional `else` executed if all other conditions are false

`then`/`else`/`elseif` keywords are all optional but good for readability




```
if {$area < $area_desired} then {
        puts "Desired area constraint met."
} elseif {$area < $area_max} then {
        puts "Maximum area constraint met."
} else {
        puts "Area constraints VIOLATED."
}
```

Condition

`if` body

`elseif` body

`else` body

15

cadence

The conditions of the `if` are evaluated in a Boolean sense meaning it is either true or false. Since Tcl does not have data types (remember – everything is a string), there are "things" that are considered to be true and "things" that are considered to be false.

- Numbers:
  - Every "non zero" number (1,2,-3.2,…) is true
  - Only 0 or 0.0 are false.
- Non number strings:
  - true, yes, on are true. (All are caseless!)
  - false, no, off are false. (All are caseless!)
  - All other strings raise an exception.

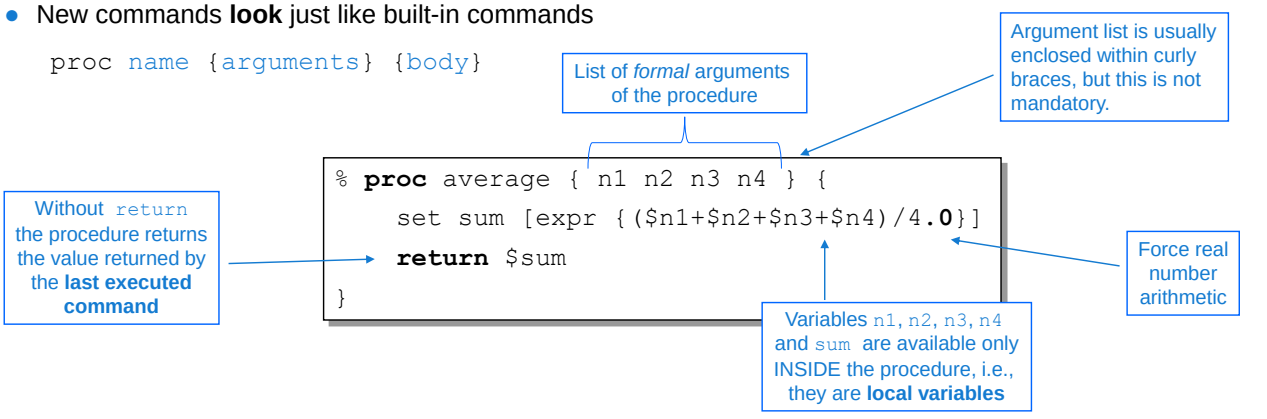
The `then` `elseif` and `else` keywords are optional. However they improve the code readability and are recommended.

Basic Procedures: Declaration

Use the `proc` command to create new Tcl commands.

- New commands **look** just like built-in commands

```
proc name {arguments} {body}
```



- There is a single global scope for the procedure name.
  - Usable/Visible everywhere (no local declaration).
  - A new procedure definition overrides any existing procedure with the same name without checking.

16

cadence

Syntax: proc <proc name> <argument list> <proc body>

`info body` returns the body of a procedure:

```
% info body average
    set sum [expr {($n1+$n2+$n3+$n4)/4.0}]
    return $sum
```

`info args` returns the arguments of a procedure:

```
% info args average
      n1 n2 n3 n4
```

`info commands` lists all built-in plus all user-defined commands/procedures:

```
% info commands a*
auto_load_index auto_import auto_execok average array auto_mkindex
auto_reset auto_qualify auto_load append after
auto_mkindex_old
```

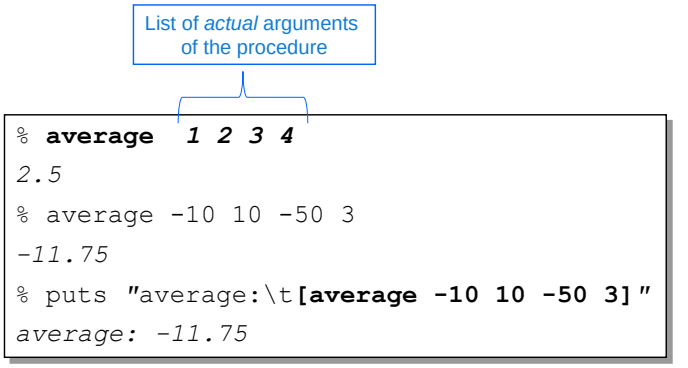
Basic Procedures: Call

Use the `proc` command to create new Tcl commands.

- New commands look and **behave** just like built-in commands.

  `% name arguments`

- Number of *actual* arguments must match the number of *formal* arguments.



Number of actual arguments must match the number of formal arguments.

Examples

```
% average 1 2
wrong # args: should be "average n1 n2 n3 n4"
% average 1 2 3 4 5
wrong # args: should be "average n1 n2 n3 n4"
```

Looping: `for`

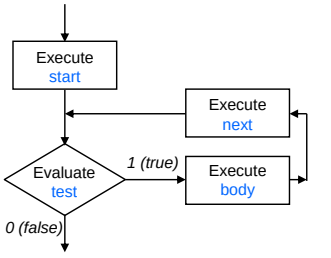
Use `for` to execute commands a specified number of times.

```
for {start} {test} {next} {body}
```

`test` is evaluated in the same way as the `expr` expression

Enclose `start`, `test`, `next` and `body` in `{}`

Example: Extract the 4 least significant bits from a positive integer number between 0 and 15




```
set a 9;
set b "";

for {set i 0} {$i < 4} {incr i} {
    set bit [expr ($a >> $i) & 0x1];
    set b "$bit$b";
}
puts "$b";

1001
```

Start Test Next Body

cadence

Here are the operators used in the example:

< less than

`incr` increment

>> right-shift

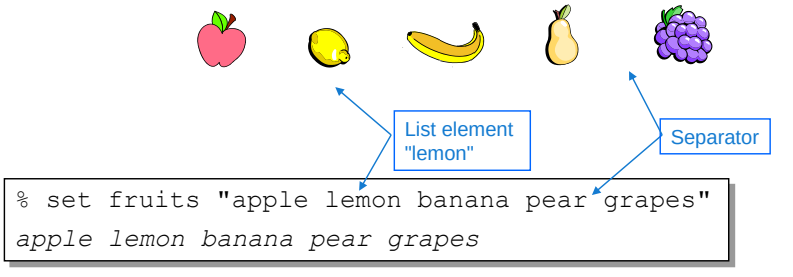
`&` logical AND

What Is a List?

A list is a collection of ORDERED elements.

- List elements are strings.
  - Can represent anything (string values, other lists, other Tcl data structures, etc.)
- List elements are separated by whitespace.
  - Spaces, tabs, newlines

Tcl list of 5 elements:



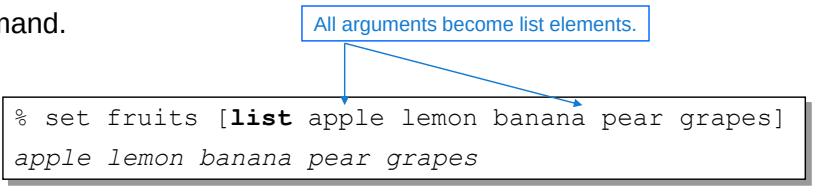
cadence

This page does not contain notes.

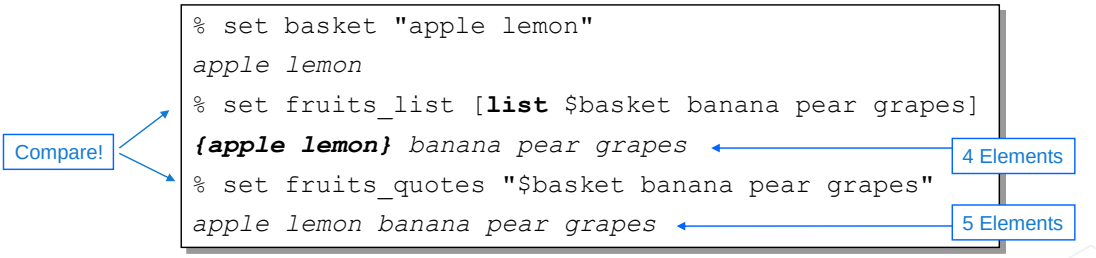
Building Lists with the `list` Command

You can build lists with `""` (or `{}`)

Or use the `list` command.



The `list` command respects the list grouping, while `""` removes one level of grouping.



If a variable added to a list using the list command is itself a list, then the list command adds the list from the variable as a single element, thus creating a nested list.

```
list ?arg arg ...?
```

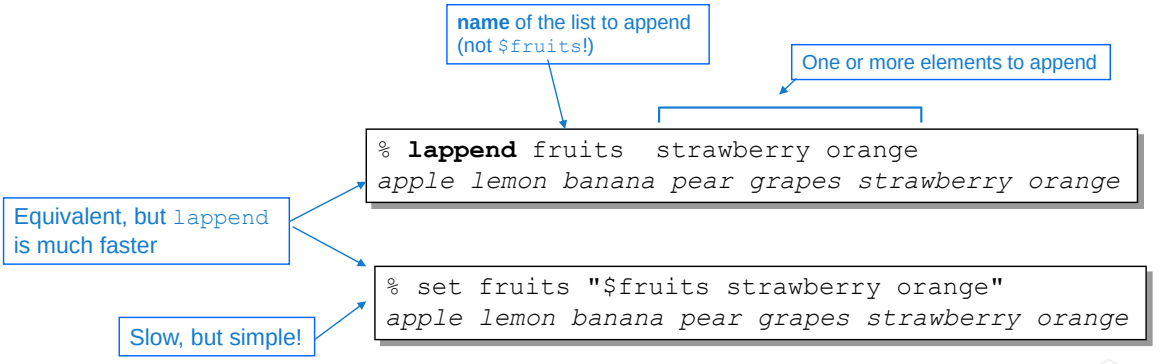
If a variable list is added using double quotes, the variable list is expanded to create multiple elements in the resulting single list.

Building Lists with the lappend Command

Use `lappend` to insert new elements at the END of a list

- Optimized for speed.
- Creates the list variable if it does not exist.

Use `lappend` when building long lists!



cadence

Syntax

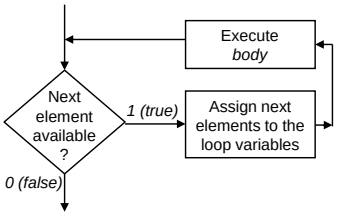
```
lappend varName ?value value ...?
```

`lappend` is significantly faster than substitution when constructing long lists.

Looping over List Elements: `foreach`

Use `foreach` to execute the loop body for each element in a list.

- Elements are processed from left to right.
- Number of iterations is equal to the number of elements.



Loop variable name

List of elements to iterate through

```
% set lib_cells "INV AND OR"
INV AND OR

% foreach cell $lib_cells {
    puts "Found library cell: $cell"
}
Found library cell: INV
Found library cell: AND
Found library cell: OR
%
```

Body

cadence

Syntax

```
foreach varName list body
```

Tcl Example: Declaration

Print the variables with value x (unknown) at a given time:

Find all registers in design and return list of absolute pathnames

```
proc p_xreg {} {
  set l_reg [ find -recursive all -absolute -internals -registers * ]
  set l_xlist "";
  foreach i $l_reg {                  # Loop over all the variables
    set v_reg [ value $i ];           # Get the value of the signal
    set isValueX [ regexp x $v_reg ]; # regexp returns 1 if value contains x
    if { $isValueX} {
      puts "$i = $v_reg";             # Print a message
      lappend l_xlist $i;             # Save x register for further processing
    }
  }
  puts "Complete list of var with x: $l_xlist";
  return $l_xlist;
}
```

xregfind.tcl

File containing procedure declaration

cadence

The `find` command is an Xcelium built-in Tcl command, used here with the following options:

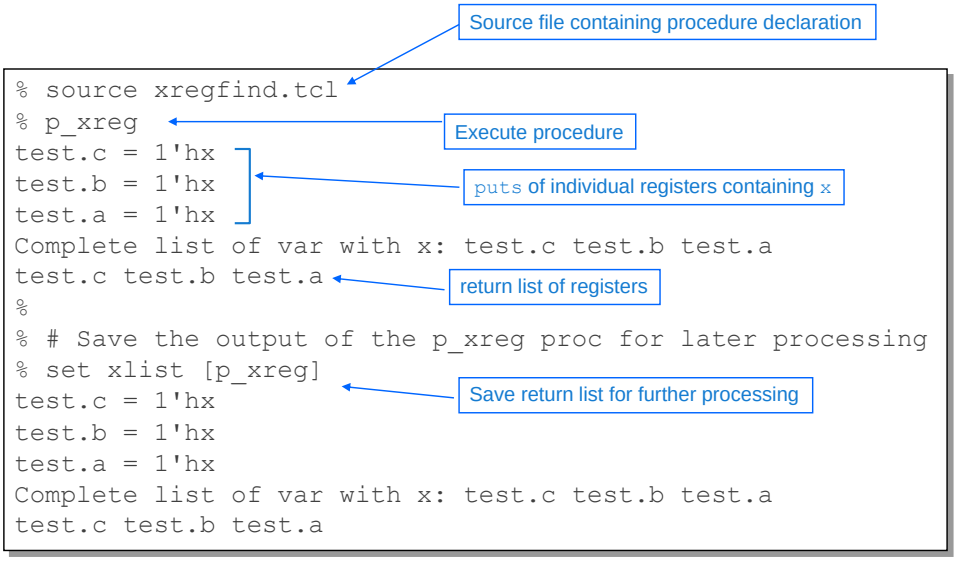
- `-recursive all` recursively search all levels of design hierarchy
- `- absolute` return absolute pathnames to found objects
- `-internals`
  `-registers` search for internal Verilog registers only. Excludes ports.
- `*` Object name to search for. `*` is a wildcard.

`value` returns the current value of the specified object

`regexp` is a regular expression search. Here looking for any `x` in the register value.

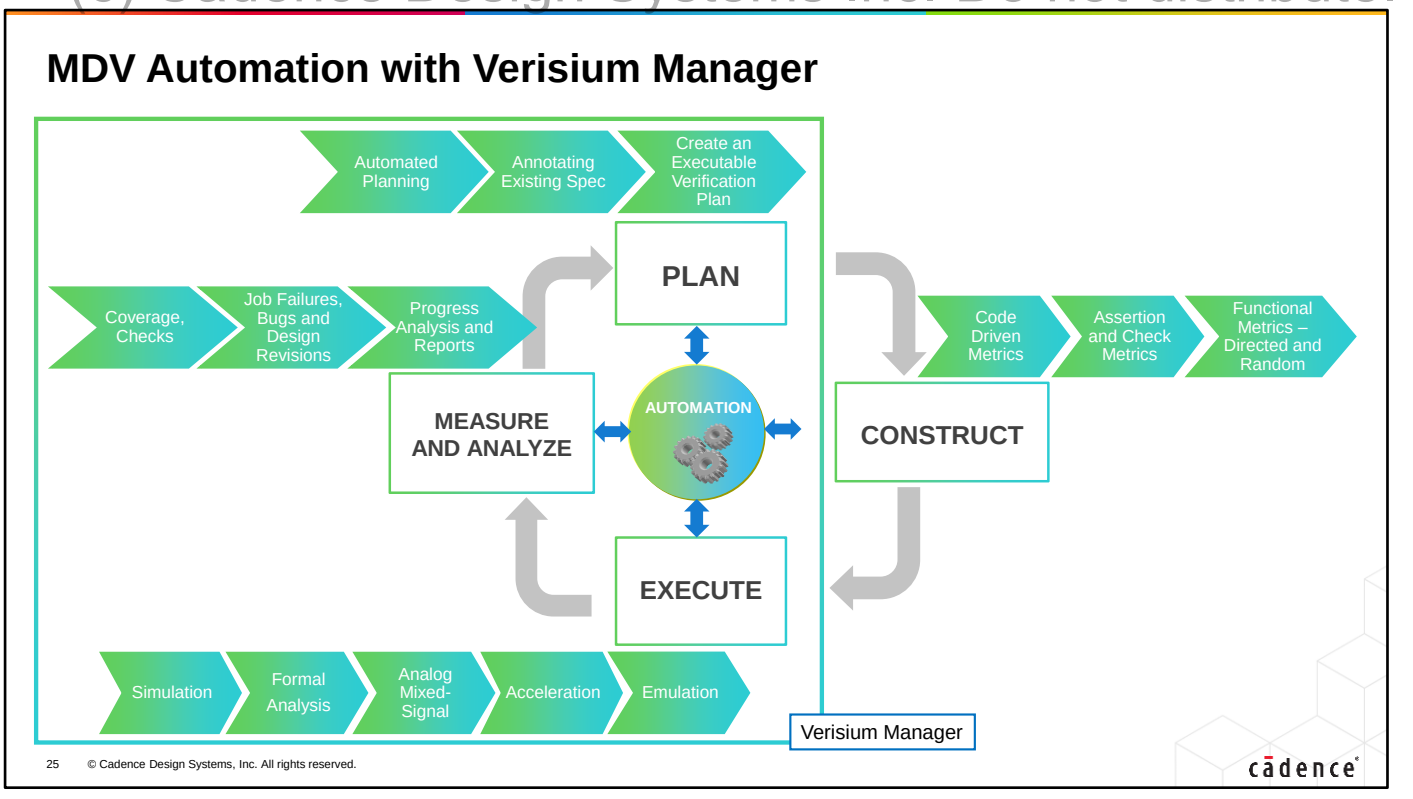
The procedure `p_xreg` prints the absolute pathname of any register containing an unknown value at the time of execution and returns a list (`l_xlist`) of all the register pathnames.

Tcl Example: Use and Results



cadence

This page does not contain notes.



The Verisium Manager tool surrounds the whole MDV Methodology except the construction phase (testbench and verification infrastructure creation). Verisium Manager helps us to automate each phase of the MDV process, using a sophisticated GUI.

Plan: Automated verification planning by either annotating existing specifications with verification intent or creating an executable verification plan using team member inputs captured in a spreadsheet.

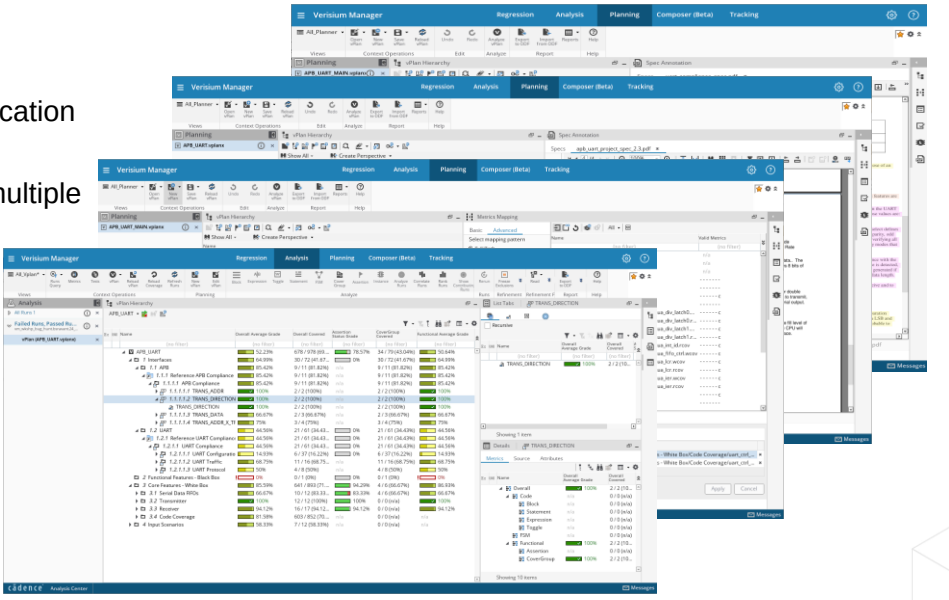
Construct: Typically a UVM testbench to verify the RTL DUT using constrained randomization to apply stimulus, coverage collection to measure what has and has not been tested, and assertions and checkers to confirm correct behavior and capture bad behavior.

Execution: Invokes multiple verification simulations using different tests and including formal analysis, analog mixed-signal, acceleration, and emulation execution engines. Collates results, failures, coverage, and assertion output from multiple verification engines.

Measurement/Analysis: MDV is used to measure real-time progress with on-demand information, as well as to determine when high-quality verification closure is achieved. Coverage, checks, and assertions provide the verification-specific metrics used to determine closure. Verification job failures, bugs, and design revisions all provide insight into the actual status of a project. Progress analysis and reports help the verification team make adjustments to their resource allocation (people and tools) to reach closure more efficiently and measure closure more accurately.

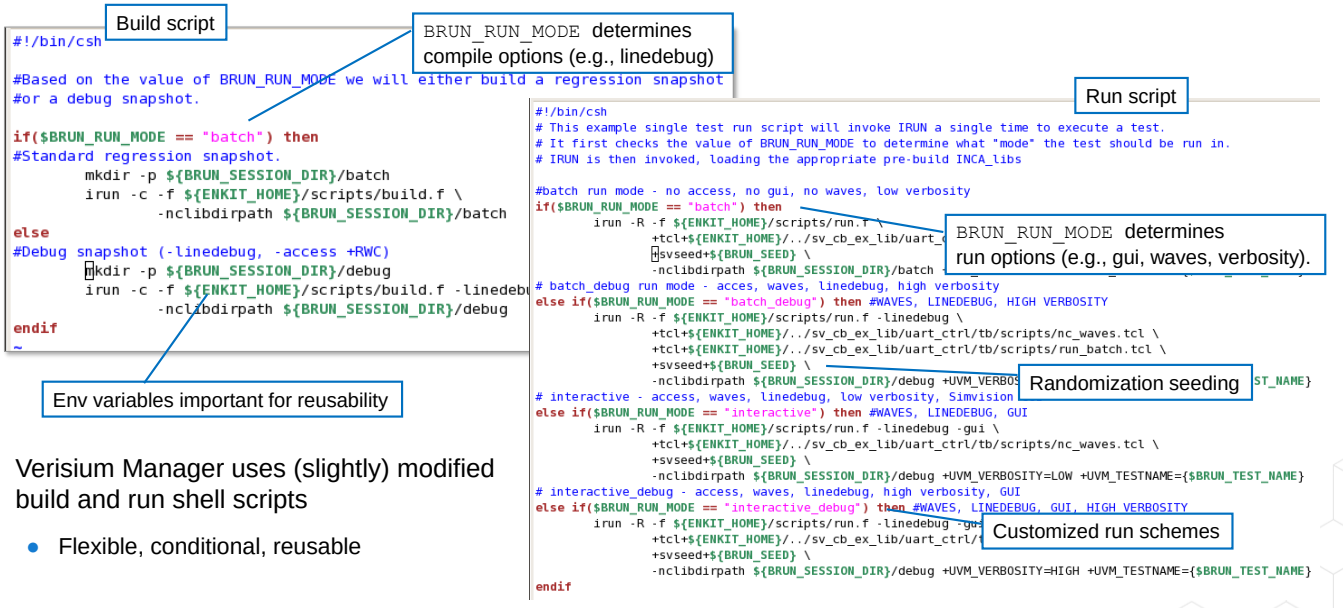
Automated Verification Planning

- Create plan elements
- Link to functional specification
- Link tests to plan from multiple verification sources
- Analyze results at the feature level with personalized views



Verification planning or creating a verification plan is the first task in your verification flow and automated verification planning is the success story of the Cadence Verisium Manager tool. Let's look at some of the basic tasks; it helps you with automated verification planning. It helps to create plan elements. It links those elements to the functional specification. It links the test bench and the tests to plan which you have created from multiple verification sources like the emulation, acceleration, simulation as well as the formal engines. It helps to finally analyze all these results at the feature level with your own settings and personalized views.

Build and Run Scripts



Verisium Manager still relies on scripts to build and run simulations. Scripts should be flexible, conditional, and reusable, e.g. using environmental variables rather than hardwired directory pathnames.

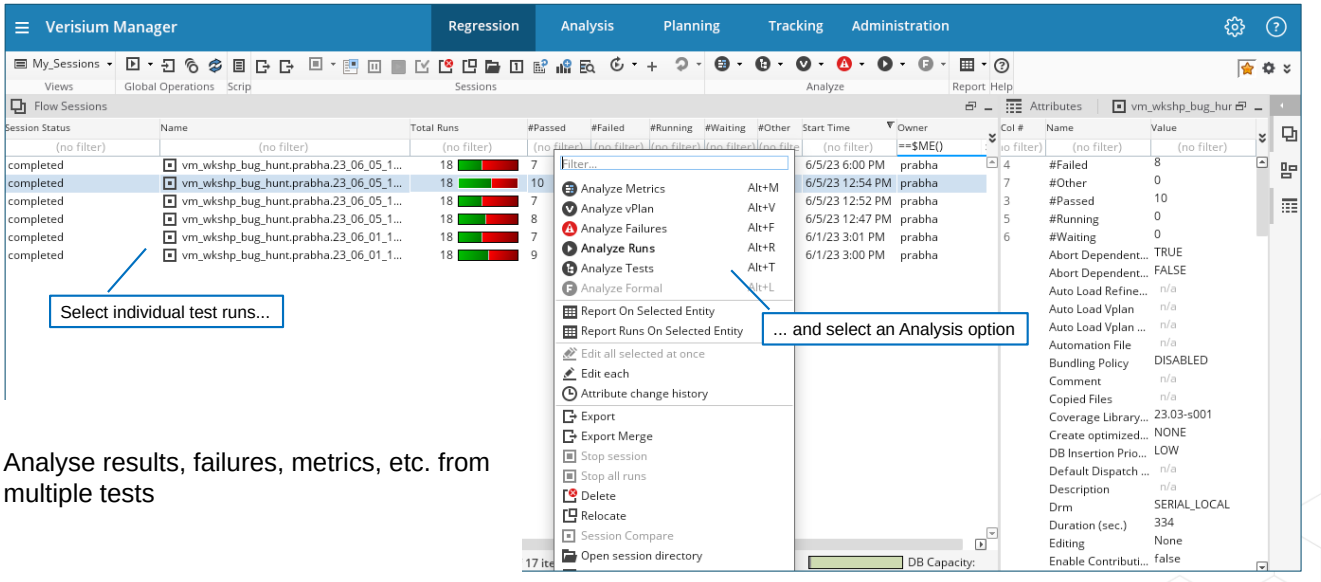
A build script modified for Verisium Manager usually does the following:

- Compiles files in the correct order, managing dependencies.
- Conditionally builds a simulation snapshot according to parameters (e.g. batch-mode, interactive).

A run script modified to work with Verisium Manager usually does the following:

- Sets up the run environment.
- Defines run variables.
- Defines which coverage and check data to be collected in Unicov format.
- Defines run modes.
- Invokes a run.

Simulation Run Analysis



Analyse results, failures, metrics, etc. from multiple tests

What can I do with the results?

Failure analysis

- Quickly find, debug, and fix failures

Coverage analysis

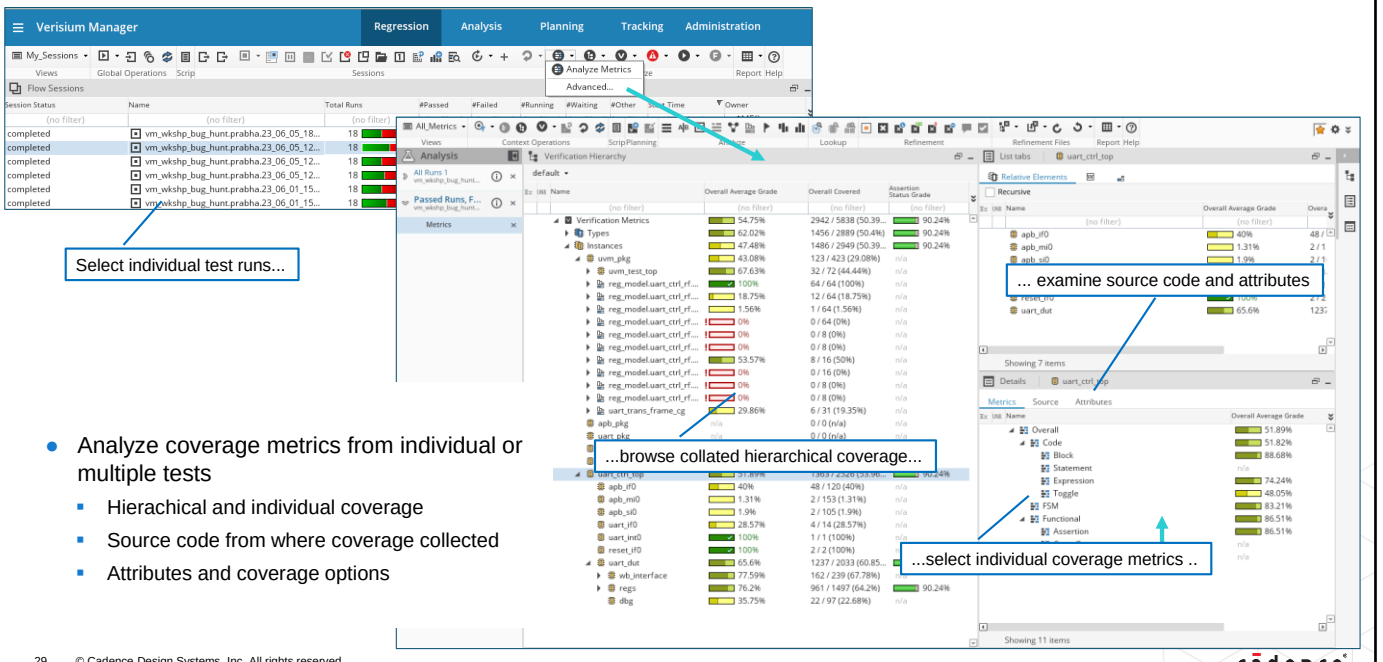
- Quickly find, debug, and fill coverage holes
- Report status and progress

Reporting

- Regression results
- Plan-based reporting

Charting

Metrics Analysis View



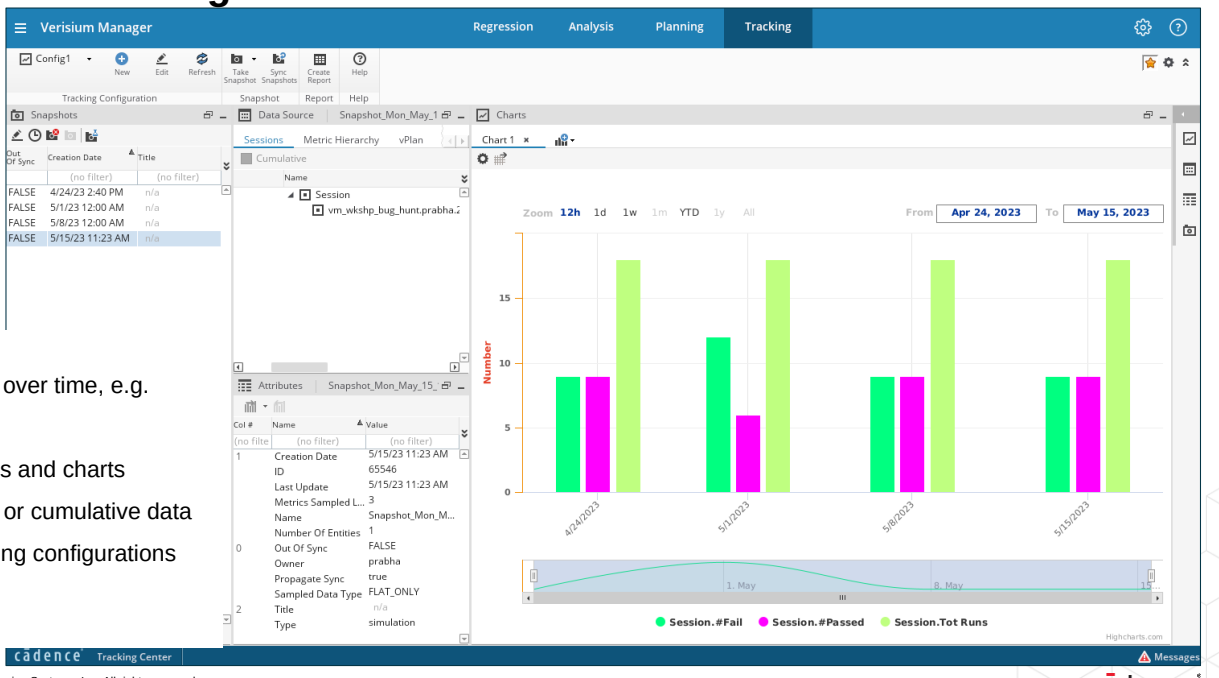
- Analyze coverage metrics from individual or multiple tests
  - Hierachical and individual coverage
  - Source code from where coverage collected
  - Attributes and coverage options

29

cadence

This page does not contain notes.

Analysis Tracking Center



Track data over time, e.g.

- Metrics

With graphs and charts

Snapshots or cumulative data

Build tracking configurations

The Tracking center allows you to track data over time. Using the Tracking center, you can track progress of sessions, tests, metrics, and verification plans. You can also generate charts and tracking reports.

The Tracking center has following toolbars:

- Tracking Configuration
- Snapshot
- Report
- Cumulative

Creating a tracking configuration includes defining configuration name, owner, specifying the metrics to be sampled, vPlan to be sampled, and defining the sessions that will be read at the time of taking the snapshot.

After creating a configuration, you can take snapshots at regular intervals (every day, every week, or a specific milestone) based on your requirements.

By tracking the cumulative data, you can easily follow the project's progress and easily find the problematic points in the project. For example, you can identify the areas in the metrics model that are not covered in any of the snapshots.

After taking snapshots, you can generate charts to track the configuration status.

Summary

Scripting is essential in verification

- To compile and build test environments
- To control simulation runs for consistency
- To save and restore debugging environments

Shell scripts are common for compilation, and Tcl is used for tool control

Automation manages multiple simulation runs

- Schedules runs, and selects tests
- Collates data for analysis

There are many options, from user-defined scripts to tools such as Verisium Manager

cadence

This page does not contain notes.